



EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 4

UML Building Blocks, Rules, Mechanisms and Architecture

PROGRAMME

BSc Information Communication and Technology

COURSE CODE / YEAR

BICT 3202 · Year 3

LECTURER

Francis Fweta — ICT Lecturer

DELIVERY

2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 4 of 16



Contents

1. Week 4 Introduction	3
2. Learning Outcomes	3
3. A Simple Way to Understand UML	3
4. UML Building Blocks	4
5. Structural Things	4
6. Behavioural, Grouping and Annotational Things	5
7. Relationships in UML	6
8. UML Diagrams	7
9. Rules of UML	8
10. Mechanisms of UML	9
11. UML Architecture — a Deeper Look	11
12. Model vs Diagram — Multiple Views	12
13. Common Student Mistakes	13
14. Case Study — University E-Learning System	14
15. Practical & Tutorial Activities	15
16. Independent Learning (10 Hours)	16
17. Week 4 Summary	16
18. Examination-Style Questions	17
19. References and Further Reading	17

1. Week 4 Introduction

Week 3 introduced UML and answered the basic question: **what is UML and why do we need it?** UML is a standardised modelling language used to visualise, specify and document systems; the current formal OMG specification is **UML 2.5.1**, which defines the language through abstract syntax, semantics, notation and examples.

This week we go one level deeper. The course outline specifically requires the **building blocks of UML**, the **rules of UML**, the **mechanisms of UML** and **UML architecture**. By the end of Week 4, students should understand what UML is made of, how its notation works, how models are governed by rules, and how UML is organised conceptually.

2. Learning Outcomes

By the end of this week, a student should be able to:

- Explain the building blocks of UML and identify different types of UML elements.
- Explain structural, behavioural, grouping and annotational things.
- Explain relationships and distinguish association, dependency, generalization and realization.
- Explain the role of UML diagrams and the structural vs behavioural distinction.
- Explain UML notation (syntax), semantics and well-formedness.
- Explain specifications, adornments and common divisions.
- Explain extensibility mechanisms — stereotypes, tagged values, constraints and profiles.
- Explain UML's layered architecture and distinguish M0, M1, M2 and M3.
- Apply these concepts to a practical university information system.
- Identify common mistakes students make when drawing UML.

3. A Simple Way to Understand UML

Before technical definitions, imagine we want to build a University Student Management System. We need to answer several questions: **what exists?** (Student, Lecturer, Course, Registration, Payment, Result); **how are things related?** (Student — registers for — Course); **what can users do?** (Register Course, View Results, Pay Fees); **what happens?** (Login → Select → Check Prerequisite → Register → Confirmation); and **how do objects communicate?** (Student → Registration System → Database).

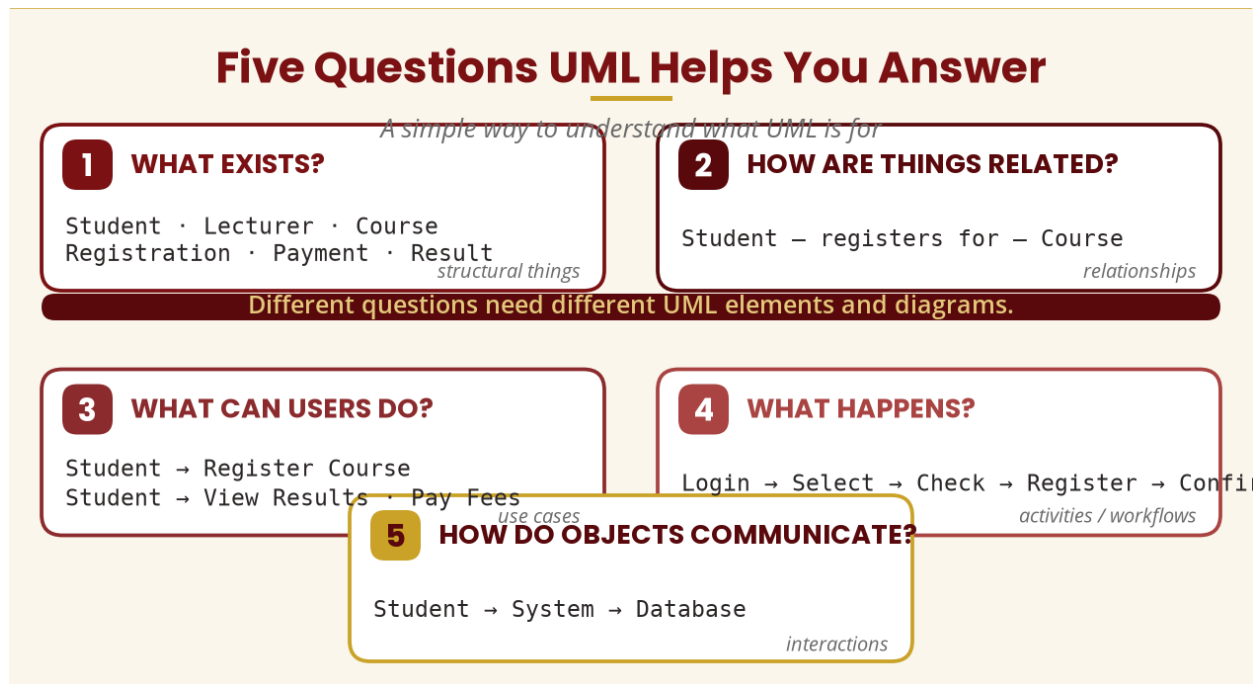


Figure 1 — Five questions UML helps you answer

4. UML Building Blocks

A useful way to understand UML is to divide its building blocks into **Things**, **Relationships** and **Diagrams**. The traditional treatment further divides “things” into **structural**, **behavioural**, **grouping** and **annotational** things — a classification that prevents students from seeing UML as a collection of random shapes.

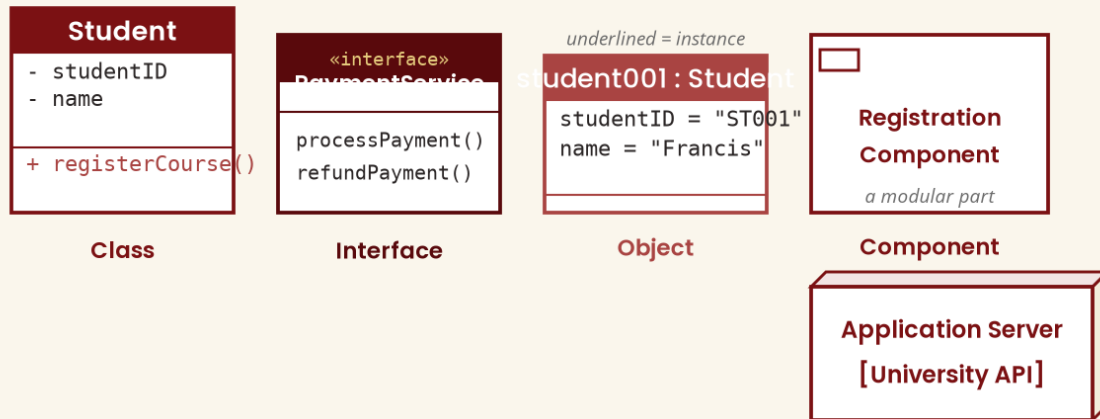
5. Structural Things

Structural things represent the relatively static parts of a system — ask “**What exists in the system?**” Examples include **class**, **interface**, **object**, **component** and **node**.

A **class** describes a type of object, shown as a three-section box (name, attributes, operations). An **object** is a particular instance of a class — written `student001 : Student` with an underlined name. An **interface** defines a set of operations that another element agrees to provide; different implementations (BankPayment, MobileMoney, CardPayment) can realise it. A **component** is a modular part of a system with a clear responsibility (connecting to Week 2’s modularity). A **node** is a computational resource or execution environment — a mobile phone, an application server or a database server — used in deployment diagrams.

Structural Things — What Exists?

The relatively static parts of a model: Class · Interface · Object · Component · Node

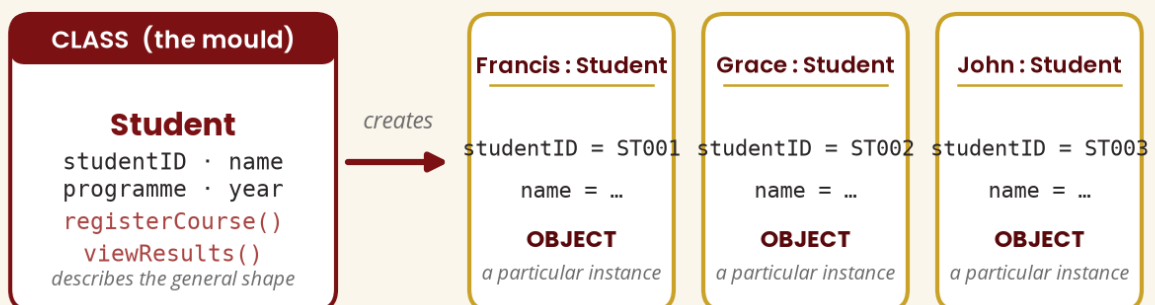


Structural things answer "what exists?" — behavioural things answer "what happens?".

Figure 2 — The five structural things

Class vs Object — the Cake-Mould Analogy

A class is the mould · an object is one cake baked from it



Class = general definition · Object = particular instance — underline object names.

Figure 3 — Class vs object: the cake-mould analogy

6. Behavioural, Grouping and Annotational Things

Behavioural things answer "What happens?" and include **interactions** (communication between elements — who communicates with whom and what messages are exchanged), **activities** (workflows, such as course registration from Login to Confirmation) and **state machines** (how an object changes state — Draft → Submitted → Under Review → Approved → Completed, where a **state** plus an **event** causes a **transition**).

Grouping things organise other elements. The most important is the **package**: a university model with 150 classes is grouped into Student Management, Finance and Examination packages, making a large model manageable.

Annotational things provide explanatory information — the simplest is a **note**, which does not represent a new business object but explains something about the model (for example, “Student must satisfy all course prerequisites before registration”).

7. Relationships in UML

A relationship tells us how model elements are connected. The four important relationships are **association**, **dependency**, **generalization** and **realization** — and they must not be confused with one another.

Association — “these elements are related” (Student ——— Course, named “registers for”).

Dependency — “this element uses that element” (ReportGenerator - - - > Payment); if Payment changes, ReportGenerator may be affected.

Generalization — “this is a specialised type of that” (Student and Lecturer are specialised forms of User; a Car is a Vehicle but a Vehicle is not necessarily a Car).

Realization — “this element fulfils the contract” (MobilePayment realises the PaymentService interface; the interface is the contract, the implementation fulfils it).

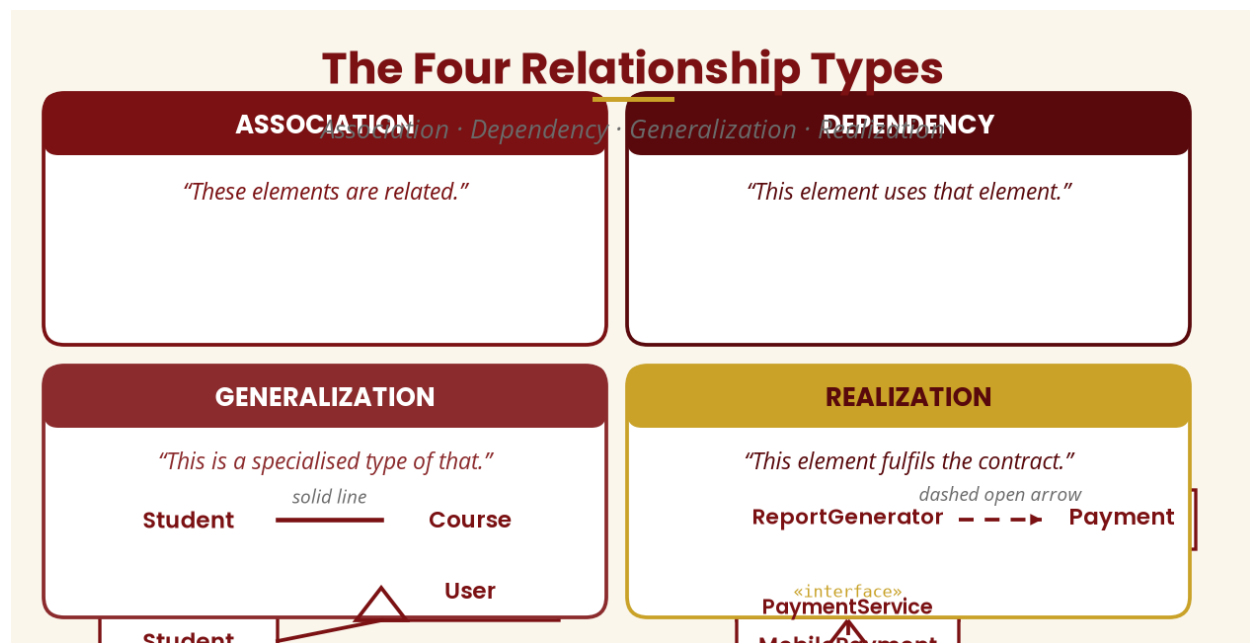


Figure 4 — The four relationship types with their notation

AN IMPORTANT OOAD INSIGHT

Student 1 ——— 0..* Course says one Student relates to zero or many Courses. But in a realistic system the relationship may need its own **Registration** class, because registration has information of its own — registrationDate, status, semester, grade. **When a relationship has important data or behaviour, it may need to become a class.**

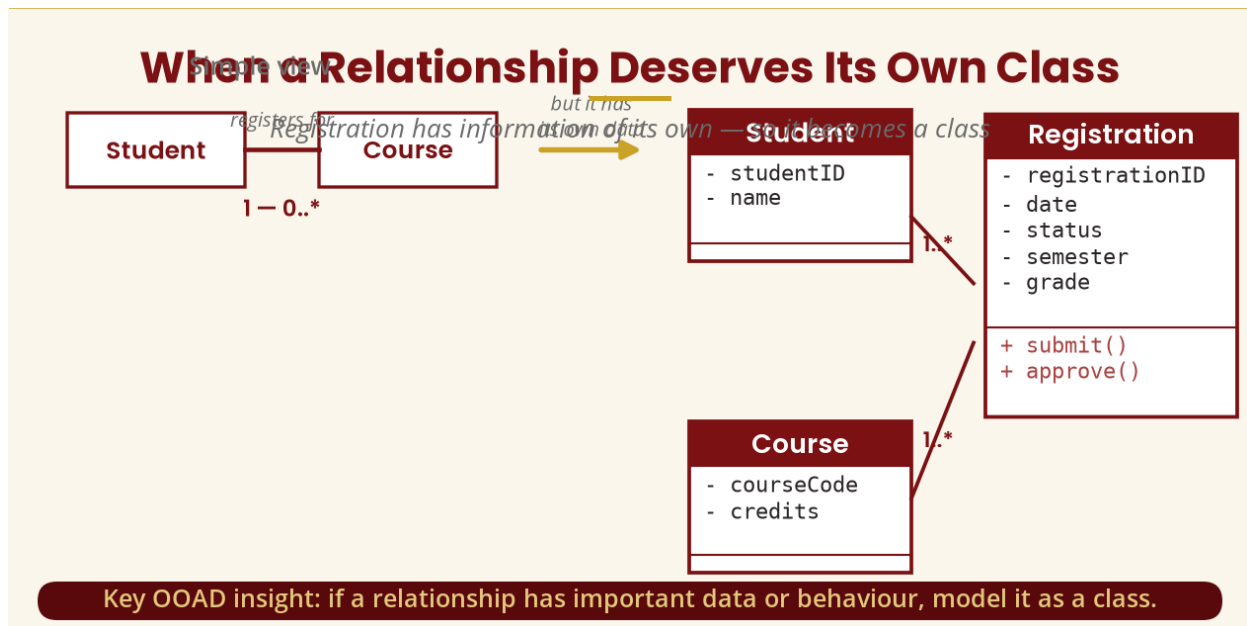


Figure 5 — When a relationship deserves its own class

8. UML Diagrams

A diagram is a particular visual view of a model — **not necessarily the whole system**. Structural diagrams (class, object, component, composite structure, package, deployment) ask “**What exists?**”; behavioural diagrams (use case, activity, state machine) ask “**What happens?**”; interaction diagrams (sequence, communication, interaction overview, timing) are a specialised behavioural family.

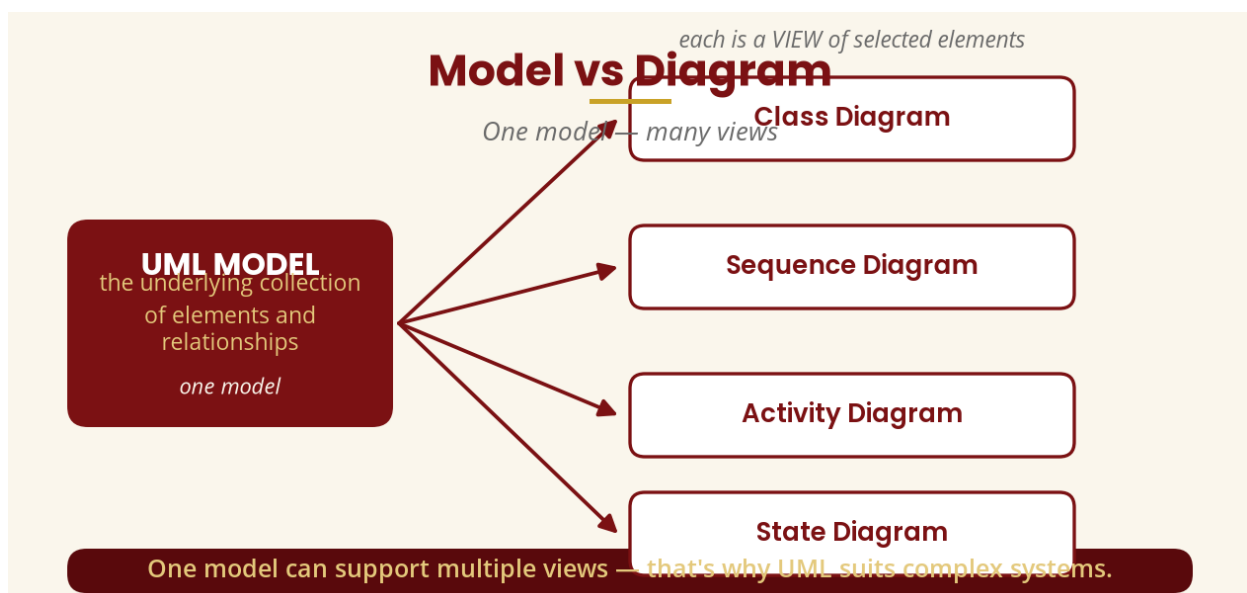


Figure 6 — One model, many diagrams

9. Rules of UML

Students sometimes think “if I draw boxes and arrows, I have created UML.” Not necessarily — UML has defined concepts, notation and semantics. **Syntax** concerns how something is represented (how a class box is drawn); **semantics** concerns what it means (a line between Student and Course is not decoration but a specific relationship).

If a student draws Student  Course and says “the diamond means they are connected”, the lecturer must ask whether they understand what that diamond means — it has a specific UML meaning (aggregation/composition). Symbols should never be chosen because “they look nice”. UML notation must be learned together with its meaning.

WELL-FORMED MODELS

A well-formed model follows the relevant rules. A model can fail as a **notation error** (the symbol is drawn incorrectly), a **semantic error** (the symbol is recognisable but communicates the wrong meaning), or a **requirement error** (valid UML, but not what the system actually needs). A perfectly drawn diagram can still describe the wrong system.

Three Ways a Model Can Fail

Notation · Semantics · Requirements

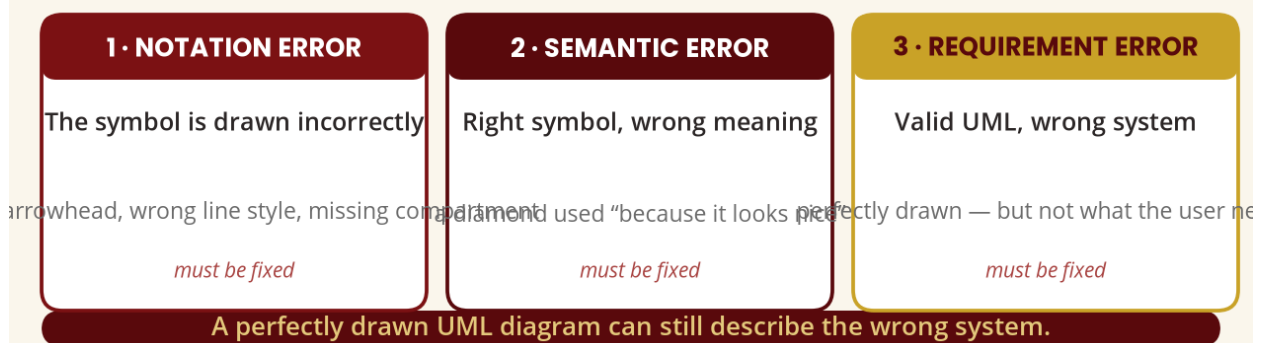


Figure 7 — Three ways a model can fail

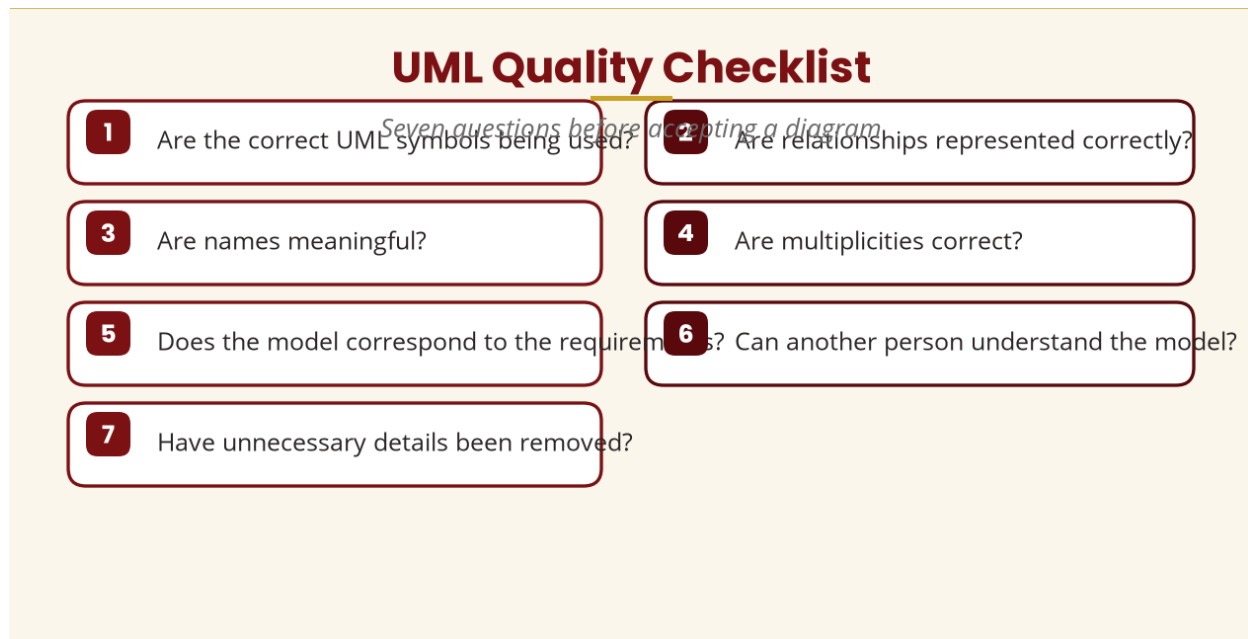


Figure 8 — The seven-question UML quality checklist

10. Mechanisms of UML

Traditional UML teaching introduces four common mechanisms: **specifications**, **adornments**, **common divisions** and **extensibility mechanisms**.

Specification: a diagram is a view of a richer underlying model — the Student box does not show every property; behind it lies the full specification (name, attributes, operations, relationships). **Adornments:** extra information attached to a basic element — the + and - visibility markers, multiplicity, constraints and modifiers. **Common divisions:** paired distinctions such as class/object (type vs instance — a cake mould vs an individual cake) and interface/implementation (contract vs behaviour).

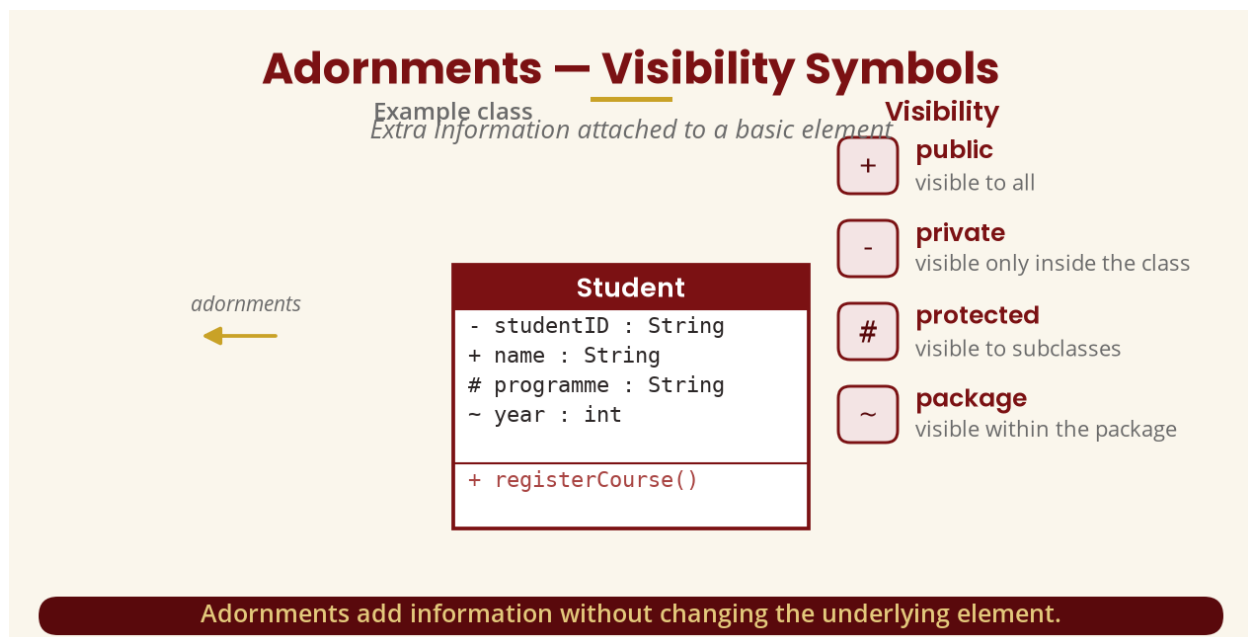


Figure 9 — Adornments: visibility symbols

VISIBILITY SYMBOLS

+ public · – private · # protected · ~ package

Extensibility mechanisms let UML be customised for domains such as banking, healthcare, telecommunications, education, manufacturing and cybersecurity. Three important concepts are **stereotypes**, **tagged values** and **constraints**.

Stereotypes — Specialised UML Meaning

Written in guillemets, a stereotype tells the reader how to treat the element



Figure 10 — Stereotypes: «entity», «controller», «boundary»

A **stereotype** gives an element a specialised meaning, written in guillemets — «entity» Student, «controller» RegistrationController, «boundary» RegistrationPage. A **tagged value** adds property/metadata — {location = "Main Data Centre"}, {author = "Analysis Team"}. A **constraint** states a condition that must hold — {age ≥ 18}, {prerequisiteSatisfied = true}. Without the constraint, a Student—Course diagram shows the relationship but not the rule governing it.

Stereotype vs Tagged Value vs Constraint

Three extensibility mechanisms — easy memory trick below

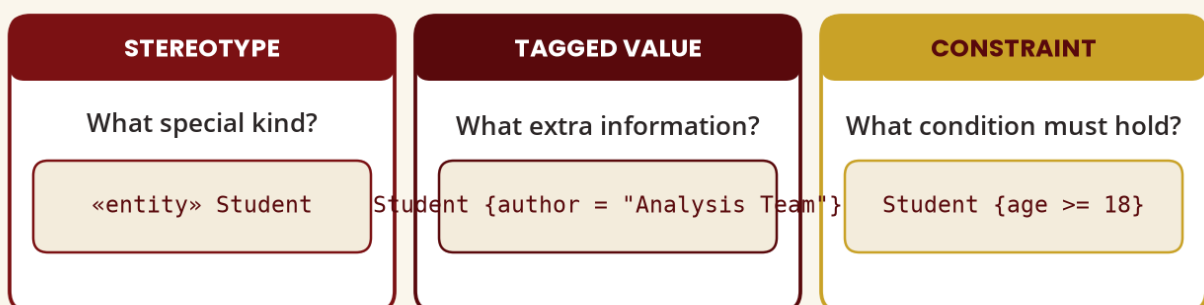


Figure 11 — Stereotype vs tagged value vs constraint

UML PROFILES

A **profile** tailors UML for a particular domain or purpose (real-time systems, business processes, security, enterprise architecture), defining stereotypes and associated extensions. **UML = general modelling language; profile = customised UML vocabulary/rules for a domain.** UML 2.5.1 includes a standard profile as a normative machine-readable artefact.

11. UML Architecture — a Deeper Look

Week 3 introduced the four layers M0–M3. This week we make them clearer.

M0 — instance level: actual objects — Francis : Student, Grace : Student, John : Student.

M1 — model level: the model we create for our system — Student, with studentID and name.

M2 — metamodel level: the UML concepts themselves — Class, Property, Operation, Association, Generalization.

M3 — meta-metamodel: the foundation used to define metamodels — the Meta Object Facility (MOF).

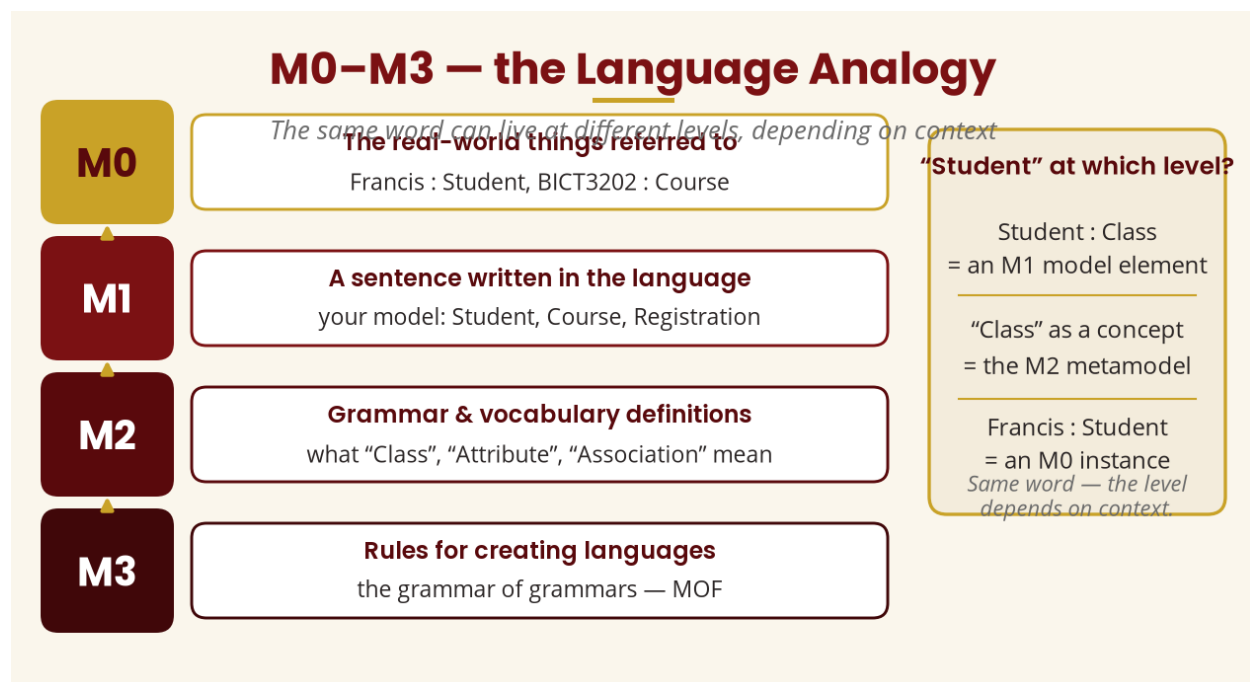


Figure 12 — The M0–M3 language analogy and context-dependent levels

Students often ask "Is Student M0, M1 or M2?" The answer depends on context: Student : Class is an M1 model element, but Class as a UML concept belongs to the M2 metamodel, while Francis : Student is an M0 instance. The same word can live at different levels.

EXAMINATION TRAP

UML architecture ≠ software architecture. UML architecture is the M3/M2/M1/M0 meta-modelling hierarchy of the language itself; software architecture is how software is organised (Presentation □ Business Logic □ Data Access □ Database). They are different concepts.

12. Model vs Diagram — Multiple Views

A **UML model** is the underlying collection of elements and relationships; a **UML diagram** is a visual view of selected elements. One model can support many views — which is why UML suits complex systems.

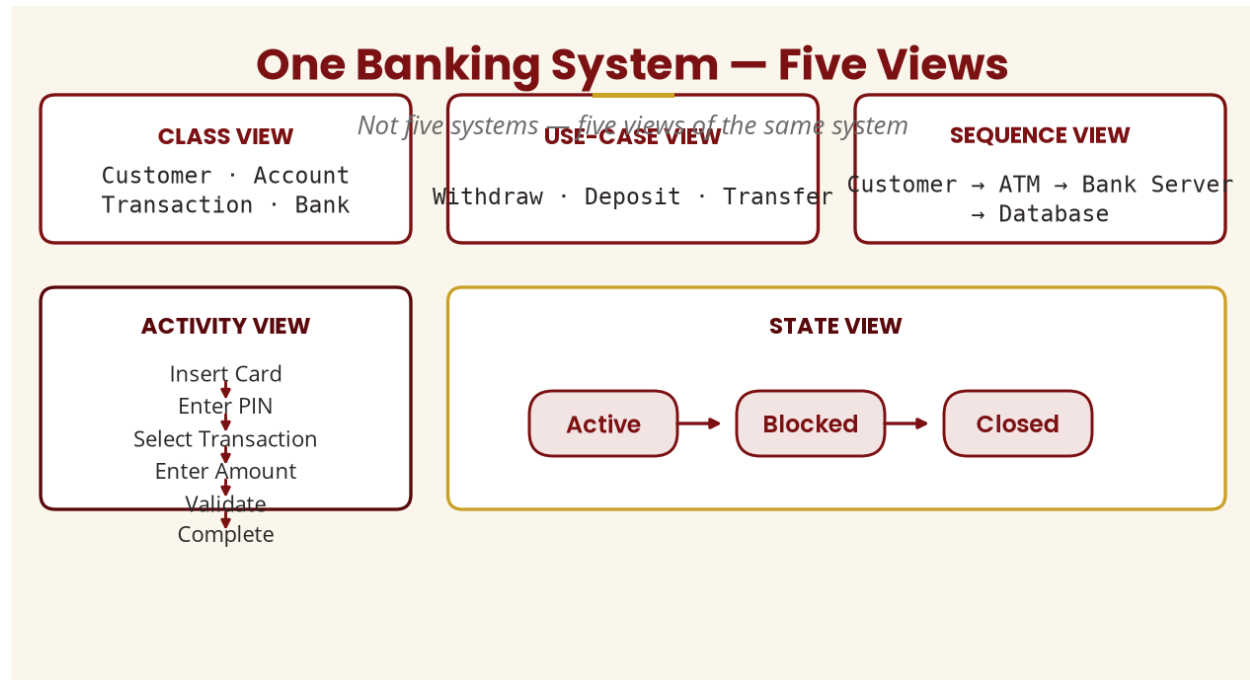


Figure 13 — One banking system, five views

UML is also a **communication tool**: when a client says “Students should register for courses”, the analyst draws (Student) ——— (Register Course); the developer asks “what if the course is full?”, the analyst updates the model, and another rule is added. A model gives stakeholders something to discuss **before** expensive implementation begins.

UML connects directly to **abstraction**: a real university has 10,000 students, 500 courses, 300 lecturers and 50 departments, but we model just Student, Course, Lecturer and Department — a manageable representation. UML also connects to **complexity management**: rather than putting 13+ classes in one unreadable diagram, we divide them into Student Management, Finance, Examination, Library and Accommodation packages.

The Power of Abstraction

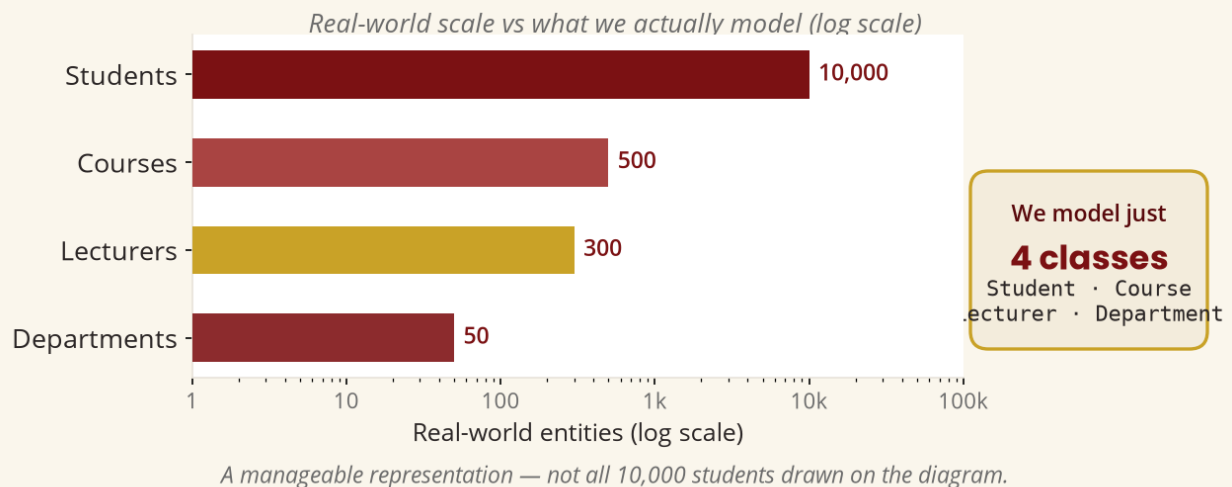
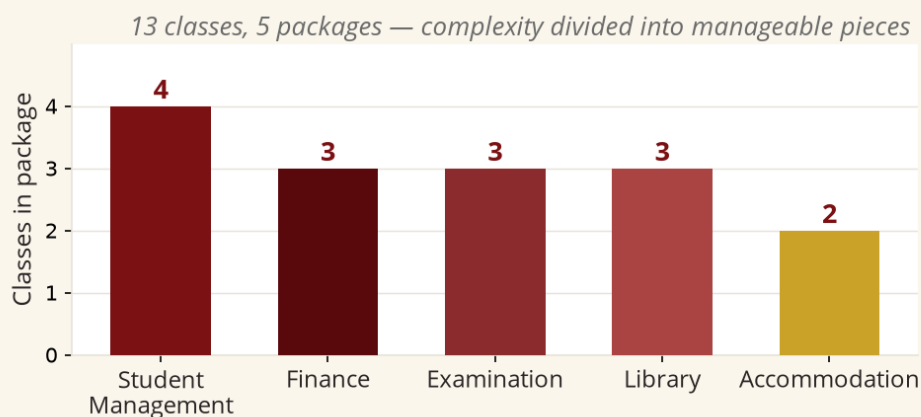


Figure 14 — The power of abstraction

Grouping Classes into Packages



Illustrative — derived from the Week 4 complexity-management example (Section 62).

Figure 15 — Grouping classes into packages

13. Common Student Mistakes

Mistake 1 — Generalization arrow points the wrong way.

If Student is a specialised type of User, the hollow triangle must point to User (the general class). A very common beginner error.

Mistake 2 — Choosing symbols by appearance.

A diamond has a specific meaning (aggregation/composition). “It looks nice” is never a valid reason.

Mistake 3 — Everything becomes a class.

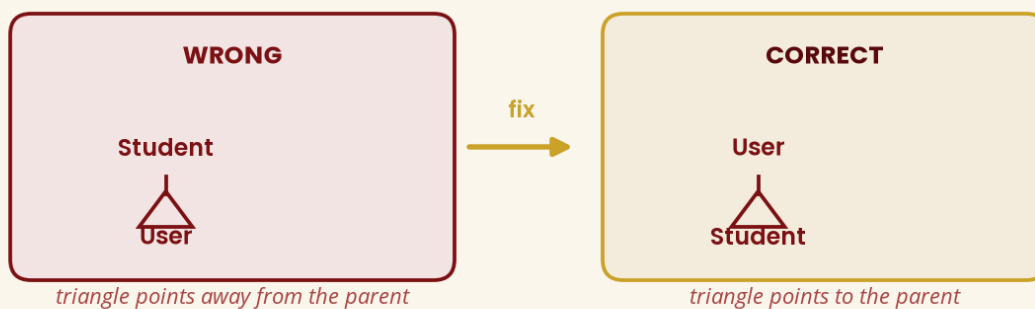
Name, StudentID, CourseName and RegistrationDate are not automatically classes. Object identification is analysis, not a grammar exercise — ask whether it has attributes, behaviour, relationships, and whether the system needs to store it.

Mistake 4 — Overloading the diagram.

Putting Student, Course, Library, Book, Hostel, Food and Transport in one diagram makes it unreadable. Separate related areas into packages/models.

Common Mistake — Generalization Direction

The hollow triangle must point to the general (parent) class



Student is a type of User — so the arrow points to User. A very common beginner error.

Figure 16 — Correct vs incorrect generalization direction

14. Case Study — University E-Learning System

Imagine a university wants an online learning platform. Students can log in, view courses, download notes, submit assignments and view grades; lecturers can upload notes, create assignments, mark assignments and publish grades.

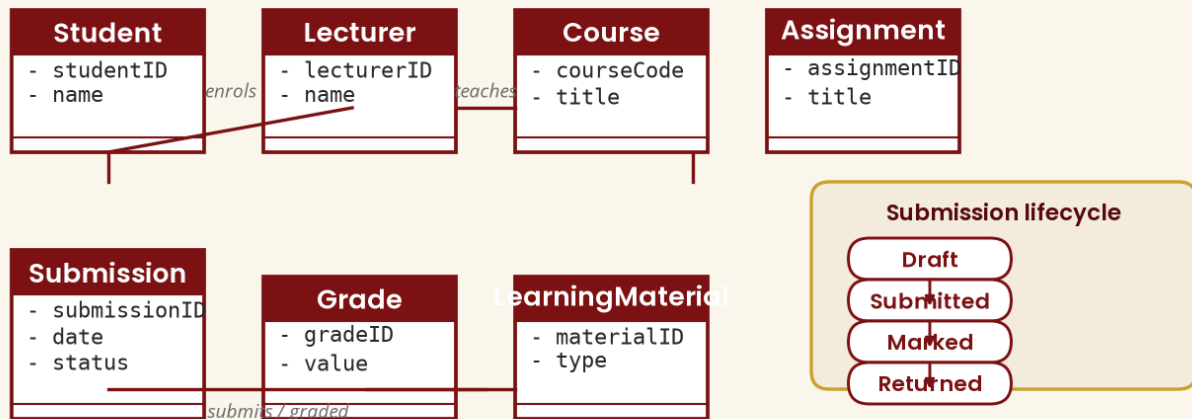
Possible classes: Student, Lecturer, Course, Assignment, Submission, Grade, LearningMaterial.

Relationships: Student—Submission, Student—Course, Lecturer—Course, Course—Assignment, Assignment—Submission.

Behaviour to model: Submit Assignment, Mark Assignment, Publish Grade. **State changes** (for a submission): Draft → Submitted → Marked → Returned.

Case Study — Exploits University E-Learning System

Candidate classes, relationships and a submission state machine



Classes + relationships + the state changes of a Submission — three UML views of one system.

Figure 17 — E-learning system: classes, relationships and the Submission lifecycle

15. Practical & Tutorial Activities

15.1 Practical — University Course Registration

Build a small model in steps: **(1)** identify classes (Student, Course, Lecturer, Registration); **(2)** identify attributes (Student: studentID, name, programme, year; Course: courseCode, courseName, creditHours; Lecturer: lecturerID, name, department; Registration: registrationID, date, status); **(3)** identify operations (registerCourse(), dropCourse(), viewResults(); assignGrade(), viewClassList()); **(4)** add relationships (Student—Registration, Registration—Course, Lecturer—Course); **(5)** add multiplicity (Student 1 — 0..* Registration, Course 1 — 0..* Registration); **(6)** add a constraint {prerequisiteSatisfied = true}; **(7)** add a stereotype where appropriate («entity» Student).

15.2 Practical — Identify the Relationship

Statement	Relationship
A Student is a type of User.	Generalization (Student $\sqsubseteq$ User)
A Registration uses information from Course.	Dependency (if genuinely usage)
A Student registers for a Course.	Association
MobilePayment implements PaymentService.	Realization

15.3 Tutorial — Find the Mistake

Given a diagram where Student points (hollow triangle) down to User, students must identify that the generalization arrow should point toward the general class — User at the top, Student below. This is the most common beginner mistake.

16. Independent Learning (10 Hours)

- **2 hours — Reading:** the relevant sections of the OMG UML 2.5.1 specification (a reference, read non-sequentially).
- **3 hours — Diagram practice:** create 2 class diagrams, 2 use-case diagrams, 1 activity diagram and 1 sequence diagram.
- **2 hours — Relationship practice:** association, dependency, generalization, realization.
- **2 hours — Case study:** model a small University Library Management System.
- **1 hour — Revision:** answer the examination questions without looking at notes.

17. Week 4 Summary

Term	Meaning
UML	Unified Modeling Language.
Class / Object	Description of a type of object / an instance of a class.
Interface	Contract defining available operations.
Component / Node	Modular system element / computational or execution resource.
Association	Structural relationship.
Dependency	Usage/dependency relationship.
Generalization	General-to-specific relationship.
Realization	Implementation of a contract.
Package / Note	Grouping mechanism / annotation.
Stereotype / Tagged value / Constraint	Specialised meaning / extra metadata / condition or rule.
Profile	UML customisation mechanism.
Syntax / Semantics	Form and notation / meaning.
M3 / M2 / M1 / M0	Meta-metamodel / metamodel / model / instance levels.

THE BIG PICTURE — WEEKS 1–4

Week 1 answered **why** the organisation needs the system; Week 2 answered **what objects exist** and how they relate; Week 3 asked **how we communicate** these models; Week 4 establishes **UML's foundations** — its elements, symbols, rules, extension mechanisms and organisation. Week 4 is the bridge between understanding UML conceptually and actually using it to perform Object-Oriented Analysis, which begins in Week 5.

18. Examination-Style Questions

Question 1 (Short essay). Explain the major building blocks of UML and give an example of each.

Question 2 (Comparison). Differentiate between association, dependency, generalization and realization, using diagrams in your answer.

Question 3 (Application). A university has Students, Lecturers, Courses and Departments; students register for courses and lecturers teach courses. Draw a UML class diagram showing classes, attributes, relationships and multiplicities.

Question 4 (Extensibility). Explain stereotype, tagged value, constraint and UML profile, with a practical example of each.

Question 5 (Architecture). Explain the four levels M3, M2, M1 and M0 using a university registration system as your example.

19. References and Further Reading

Primary authoritative reference

- Object Management Group. (2017). *Unified Modeling Language (UML), Version 2.5.1*. OMG. (Adopted December 2017 — the formal version listed by OMG.)
- Object Management Group. *UML 2.5.1 — Formal Specification* (abstract syntax, semantics, notation, examples) and *UML 2.5.1 Abstract Syntax Metamodel* (with the UML standard profile and diagram interchange metamodel).

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML*.